

Unidade IV

7 METODOLOGIAS ÁGEIS I

As metodologias ágeis são abordagens de desenvolvimento de software que promovem flexibilidade, colaboração e adaptação rápida às mudanças. Elas foram criadas como uma alternativa aos modelos tradicionais, como o Cascata e o RUP.

Embora os modelos tradicionais sejam úteis em determinados contextos, a crescente complexidade dos projetos e a necessidade de maior flexibilidade impulsionaram a busca por alternativas. As metodologias ágeis surgiram como uma resposta a essas limitações e desafios enfrentados pelas abordagens tradicionais de desenvolvimento de software.

Agora trataremos das seguintes metodologias: Manifesto Ágil, XP, Scrum e FDD.

7.1 Manifesto para Desenvolvimento Ágil de Software

Antes da criação das metodologias ágeis, a indústria de software enfrentava desafios como atrasos, custos elevados e baixa qualidade nos projetos. O Manifesto para Desenvolvimento Ágil de Software, publicado nos dias 11 a 13 de fevereiro de 2001, por Kent Beck e mais 16 notáveis desenvolvedores, foi precedido por uma série de eventos e reflexões que surgiram da necessidade de melhorar os processos de desenvolvimento de software.

Kent Beck e seus colegas se reuniram para defender as seguintes regras:

Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Por meio desse trabalho, passamos a valorizar:

Indivíduos e interações mais que processos e ferramentas.

Software em funcionamento mais do que documentação abrangente.

Colaboração com o cliente mais do que negociação de contratos.

Responder a mudanças mais que seguir um plano.

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda (Beck *et al.*, 2001).



Saiba mais

Explore o que o Copilot IA da Microsoft tem a dizer sobre o Manifesto Ágil.

Um manifesto é uma declaração pública que expressa os princípios, os valores, as intenções ou os objetivos de um grupo ou indivíduo. Geralmente, ele serve como um documento oficial que define a visão e a direção de algo, seja um movimento, uma organização ou uma ideia. Manifestos costumam ser usados para inspirar mudanças, engajar pessoas e comunicar crenças fundamentais. Um exemplo famoso inclui o Manifesto Ágil (2001), que define os princípios e os valores para o desenvolvimento ágil de software.

Compreenda melhor o exposto em:

BECK, K. *et al.* Manifesto para Desenvolvimento Ágil de Software. *Agile Manifesto*, 2001. Disponível em: <https://shre.ink/ebdl>. Acesso em: 23 jun. 2025.

Observe a figura 49. O desenvolvimento baseado em planos (desenvolvimento prescritivo) pauta-se em uma abordagem estruturada e sequencial, com etapas planejadas e documentadas antes do início do projeto. Essa metodologia usa modelos clássicos de processo, como o Cascata, que organiza o trabalho em fases distintas, incluindo levantamento de requisitos, design, codificação, testes e manutenção.

Contudo, o desenvolvimento prescritivo é valorizado pela sua capacidade de prever resultados e reduzir riscos em projetos bem definidos. É apoiado em técnicas como a modelagem estruturada e a criação de artefatos detalhados, garantindo clareza e rastreabilidade ao longo de todo o ciclo de vida do software.

Embora eficiente para sistemas com requisitos estáveis, essa abordagem enfrenta desafios em cenários mais dinâmicos, nos quais a inovação tecnológica e a evolução dos requisitos demandam maior flexibilidade. Por esse motivo, ela tem sido complementada por metodologias ágeis e iterativas que permitem maior adaptabilidade sem comprometer os princípios fundamentais de planejamento e controle da engenharia de software.

O desenvolvimento ágil é uma abordagem iterativa e incremental, com baixo índice de especificação, que prioriza a adaptação a mudanças e a entrega de valor contínuo ao cliente. Observando a figura 49, do ponto de vista da logística de serviços, vemos que a atividade omitida é a "especificação de requisitos", ou seja, foi uma grande mudança nas estratégias de serviços. O desenvolvimento ágil integrou essa fase em seus processos, aproximando mais o cliente do desenvolvimento e tornando claras e exatas as especificações, com histórias sobre o negócio, as atividades de reuniões breves e dinâmicas e as situações do desenvolvimento em tempo real. Assim, promoveu mais eficiência na comunicação entre stakeholders e desenvolvedores.

Enfim, enquanto o modelo prescritivo enfatiza a previsibilidade e o planejamento detalhado com documentação excessiva e que nem sempre são usadas, o ágil valoriza flexibilidade, colaboração e feedback constante.

O desenvolvimento prescritivo pode ser mais adequado para projetos estáveis e bem definidos, enquanto o ágil se destaca em cenários dinâmicos com interação entre indivíduos sobre processos rígidos, software funcional e sujeitos a mudanças constantes.

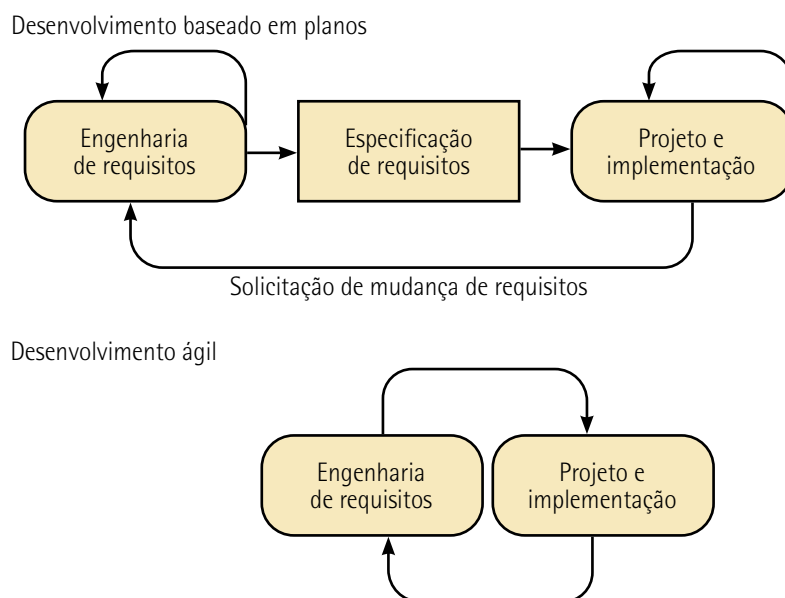


Figura 49 – Especificação dirigida com desenvolvimento ágil

Adaptada de: Sommerville (2016).

7.1.1 Princípios das metodologias ágeis

Métodos ágeis representam um conjunto de abordagens estruturadas pautadas na prática para modelagem efetiva de sistemas baseados em software, fundamentando-se em princípios de flexibilidade, colaboração e entrega incremental. É uma filosofia na qual muitas metodologias se encaixam.

Dessa forma, o desenvolvimento ágil não é uma metodologia única, mas um paradigma no qual diferentes frameworks, como XP, Scrum e FDD se encaixam; proporcionando um conjunto de diretrizes adaptáveis às necessidades específicas de cada projeto e equipe.

Guiados por valores estabelecidos no Manifesto Ágil, os métodos ágeis empregam práticas como integração contínua, desenvolvimento iterativo, refatoração de código e testes automatizados para garantir qualidade e eficiência no ciclo de vida do desenvolvimento.

As metodologias ágeis empregam uma coleção de práticas, guiadas pelos princípios apresentados no quadro 14, e valores que podem ser aplicados por profissionais de software no dia a dia.

Quadro 14 – Princípios dos métodos ágeis

Princípios	Descrição
Envolvimento do cliente	Os clientes devem estar intimamente envolvidos no processo de desenvolvimento. Seu papel é fornecer e priorizar novos RS e avaliar iterações
Entrega incremental	O software é desenvolvido em incrementos com o cliente, especificando os requisitos incluídos em cada um
Pessoas, não processos	As habilidades da equipe de desenvolvimento devem ser reconhecidas e exploradas. Membros da equipe devem desenvolver suas próprias maneiras de trabalhar, sem processos prescritivos
Aceitar as mudanças	Deve-se ter em mente que os RS vão mudar. Por isso, projete o sistema de maneira a acomodar essas mudanças
Manter a simplicidade	Tenha foco na simplicidade, tanto do software a ser desenvolvido quanto do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema

Adaptado de: Sommerville (2011).



Lembrete

As metodologias ágeis são abordagens de desenvolvimento de software que promovem flexibilidade, colaboração e adaptação rápida às mudanças.

7.2 XP

A metodologia ágil XP tem foco na melhoria contínua da qualidade do código e na capacidade de respostas às mudanças de requisitos. Ela é ideal para projetos dinâmicos, nos quais os requisitos mudam com frequência, promovendo um ambiente colaborativo e eficiente no desenvolvimento de software.

As ideias subjacentes aos métodos ágeis foram desenvolvidas mais ou menos ao mesmo tempo por várias pessoas diferentes na década de 1990. No entanto, talvez a abordagem mais significativa para mudar a cultura de desenvolvimento de software foi o desenvolvimento da Extreme Programming (XP) (Sommerville, 2016, p. 77).

Criada por Kent Beck em 1998, a XP elevou a níveis "extremos" boas práticas reconhecidas, a exemplo do desenvolvimento iterativo. Ela enfatiza valores como comunicação, simplicidade, feedback, coragem e respeito. Por exemplo, várias versões novas de um sistema podem ser desenvolvidas por diferentes programadores, integradas e testadas em um dia (Sommerville, 2016).

A figura 50 ilustra o ciclo de desenvolvimento do processo da XP para produzir um incremento do sistema. São quatro as atividades-chave da XP a serem desenvolvidas: planejamento, projeto, codificação e teste.

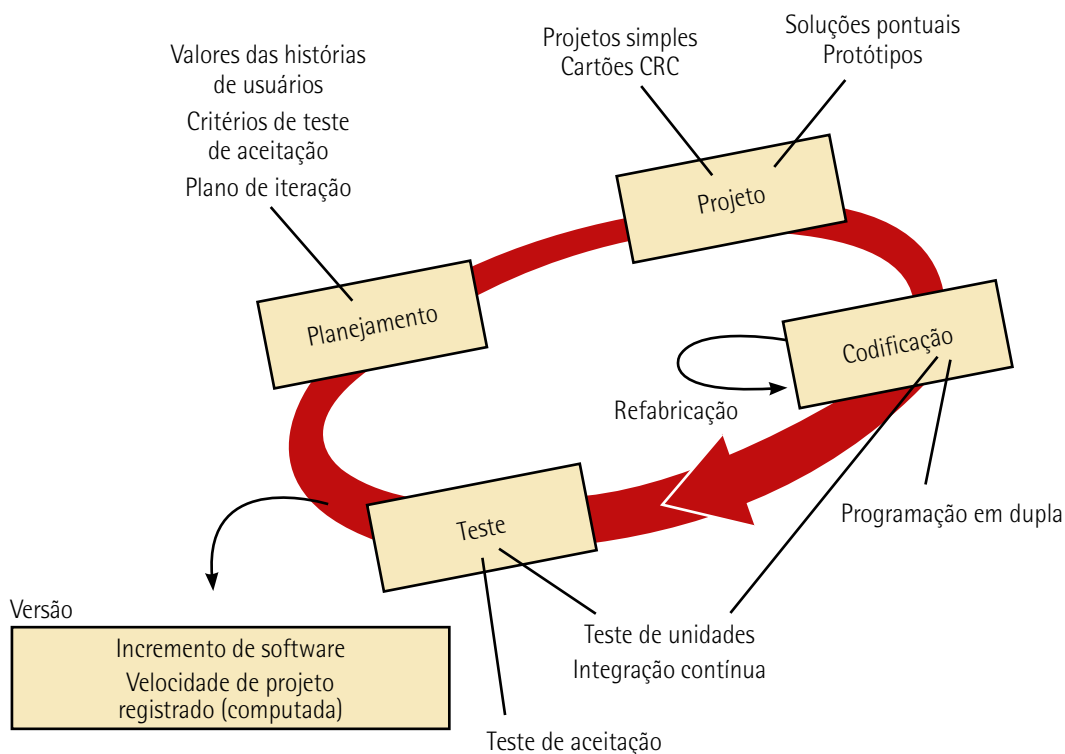


Figura 50 – O processo da XP

Adaptada de: Pressman (2021).

Planejamento

O planejamento do projeto em metodologias ágeis prioriza a eficiência e a adaptabilidade, evitando processos complexos ou demorados. Conforme o projeto avança, o planejamento incremental e iterativo permite entregas frequentes e implementação das mudanças necessárias durante todo o ciclo de desenvolvimento.

Os valores das histórias de usuários são baseados em descrições simples e claras de funcionalidades desejadas, escritas na perspectiva do usuário. Cada história é priorizada com base no valor agregado ao negócio.

Os critérios de aceitação são formulados em contratos entre stakeholders e desenvolvedores para assegurar que os requisitos estejam claros.

A XP considera o conhecimento técnico da equipe, permitindo que certos detalhes operacionais sejam omitidos, confiando na experiência dos desenvolvedores para interpretar e implementar soluções alinhadas aos objetivos do projeto. A simplicidade dessa ação evita a execução de funcionalidades desnecessárias.

O plano de iteração consiste em organizar as histórias selecionadas dos usuários para serem implementadas. A equipe trabalha para estimar o esforço necessário para cada história, garantindo

que a iteração seja realista e alcançável dentro do prazo definido, normalmente de uma a duas semanas. Nesse item, é essencial a colaboração entre stakeholders e desenvolvedores para ajustes necessários.

A fase de planejamento é conhecida como jogo do planejamento (planning game), que se inicia com a atividade "ouvir", na qual são coletados e analisados os RF e os RNF. Essa fase inclui critérios da qualidade e expectativas do software, garantindo que a equipe compreenda as demandas do cliente, a definição das funcionalidades prioritárias e tenha foco na entrega de valor contínua.

Projeto

O design de software na XP segue o princípio KIS (Keep it Simple – Preservar a Simplicidade), ou seja, prioriza soluções simples e eficazes, evitando complexidades desnecessárias.

O projeto não antecipa funcionalidades além do necessário, as mudanças e as novas necessidades são tratadas de forma iterativa ao longo do desenvolvimento. Implementações de funcionalidades baseadas em suposições futuras são desencorajadas para que o sistema permaneça enxuto e adaptável.

Nas ações são usados cartões CRC (Classe-Responsabilidade-Colaborador) para sessões de design. Esses cartões identificam e organizam classes orientadas a objetos, suas responsabilidades e colaboradores.

Em apoio ao gerenciamento do fluxo de trabalho, a XP faz uso do Kanban, que é um sistema de gestão visual que permite acompanhar o progresso das tarefas. Ele identifica gargalos e garantias de um desenvolvimento contínuo e eficiente.

Ponderando desafios técnicos ou incertezas no design, a XP recomenda a prática da prototipagem de partes do sistema antes de sua execução definitiva, possibilitando uma validação rápida de soluções antes de sua implementação ao código principal.

Codificação

Durante as iterações na XP, o código é continuamente aprimorado por meio da implementação incremental e da realização de diversos testes de unidade. A cada ciclo iterativo, o código-fonte é revisado de forma criteriosa, permitindo a identificação de oportunidades de refatoração, a correção de falhas e a incorporação de novas funcionalidades, promovendo um desenvolvimento incremental, sustentável e com alta qualidade técnica.

Um dos pilares da XP é a prática da programação em par (pair programming), que consiste na colaboração simultânea de dois desenvolvedores em uma mesma tarefa de codificação: enquanto um assume o papel de driver, escrevendo efetivamente o código, o outro atua como observador, com foco em aspectos estratégicos como detecção de erros, aderência a padrões de projeto, clareza

do código e cobertura de testes automatizados que são escritos antes da implementação do código TDD (Test-Driven Development – Desenvolvimento Orientado por Testes). Os papéis se alternam frequentemente, promovendo colaboração intensa, disseminação do conhecimento entre a equipe e aumento na qualidade do software produzido. Essa prática reflete o princípio de que “duas mentes pensam melhor do que uma” e é sustentada por estudos que demonstram ganhos em produtividade e redução de defeitos em ambientes colaborativos.

A prática de refabricação se aproxima do conceito de reengenharia de software, que envolve analisar, redesenhar e reconstruir partes significativas do sistema, geralmente para adaptar o software a novas tecnologias, arquiteturas ou requisitos que a versão atual não suporta bem. Essa prática inclui a refatoração (reestruturação) do código-fonte com o objetivo de melhorar sua legibilidade, seu desempenho e sua manutenção sem alterar a funcionalidade. Essa abordagem garante que o projeto evolua de forma sustentável, mantendo a qualidade do software ao longo de seu ciclo de vida.

Por sua vez, a prática de codificação envolve feedback rápido para que a equipe responda às mudanças requeridas de clientes e usuários de forma eficiente, bem como manter um ritmo sustentável, ou seja, trabalho sem horas extras excessivas, evitando a exaustão da equipe.

Exemplo de aplicação

O foco da prática de refatoração envolve reduzir a complexibilidade, eliminar redundâncias de código, melhorar a legibilidade e a manutenibilidade, aumentando a coesão e reduzindo o acoplamento. Veja um exemplo simples no quadro 15, que mostra a refatoração em JavaScript:

Quadro 15 – Refatoração do código-fonte

Código antes da refatoração	Código depois da refatoração
<pre>javascript function calcularDesconto(preco, tipo) { if (tipo === 'vip') { return preco * 0.8; } else if (tipo === 'comum') { return preco * 0.9; } else { return preco; } }</pre>	<pre>javascript function calcularDesconto(preco, tipo) { const descontos = { vip: 0.8, comum: 0.9, }; return preco * (descontos[tipo] 1); }</pre>
O comportamento é o mesmo, porém o código ficou mais limpo, reutilizável e fácil de manter	

Testes

O código é frequentemente integrado e testado para evitar grandes conflitos de versão. Todos os desenvolvedores podem requisitar mudanças em qualquer parte do código para promover colaboração e flexibilidade.

A atividade de testes é uma prática contínua e integrada ao ciclo de desenvolvimento. São realizados testes unitários, que validam o comportamento isolado de métodos e funções; testes de integração, que verificam a ligação entre os componentes ou módulos do software; e testes de aceitação, que asseguram que o software atenda aos requisitos do negócio determinados pelos stakeholders.



Observação

Na XP, os testes automatizados são escritos preferencialmente na fase de codificação, antes da implementação do código funcional, como defendido na prática do TDD, o que favorece o design orientado a testes, aumentando a confiabilidade da base de código. Em sua essência, os testes correspondem ao processo de apoio V&V, de forma a assegurar a estabilidade do software e viabilizar o processo de manutenção contínua com baixo risco.

Na identificação de um erro ou falha, é comum reproduzir um novo teste antes de corrigi-lo, garantindo que o erro ou falha não volte a ocorrer (regressão). São produzidos e registrados novos casos de testes para que possam ser reutilizados.

Os testes de aceitação, realizados com o envolvimento do cliente, ocorrem em ciclos curtos e são acompanhados por métricas (como pontuação ou cobertura de critérios) que ajudam a avaliar o progresso e a conformidade das entregas com os requisitos esperados.

7.2.1 Kanban

Surgiu no Japão na década de 1940 como parte do sistema de produção da Toyota, desenvolvido por Taiichi Ohno, um dos engenheiros responsáveis pelo revolucionário TPS (Toyota Production System – Sistema Toyota de Produção).

Na língua japonesa, Kanban significa "cartão". Na manufatura, ele era um método de controle de estoque e fluxo de produção que utilizava cartões físicos para sinalizar a necessidade de reposição de materiais ou peças. Isso permitia produzir apenas o necessário, no tempo certo (Just-In-Time – JIT), promovendo a filosofia de evitar desperdícios de recursos.

Na engenharia de software, Kanban é um método ágil de gerenciamento de trabalho (workflow), que visa melhorar a eficiência do fluxo de desenvolvimento e promover a entrega contínua de valor. Ele foi

adaptado para a engenharia de software como uma ferramenta visual e evolutiva para gerenciamento de processos, especialmente em ambientes que demandam alta flexibilidade e melhoria contínua.

Esse método pode ser aplicado em qualquer ramo de tarefas da engenharia de software. Como exemplo, a figura 51 mostra uma aplicação usando o processo da XP com base no Kanban, fornecendo como resultado um Kanban, que pode inclusive ser aplicado em métodos de análise de caminho crítico no caso de uma rede mais complexa.

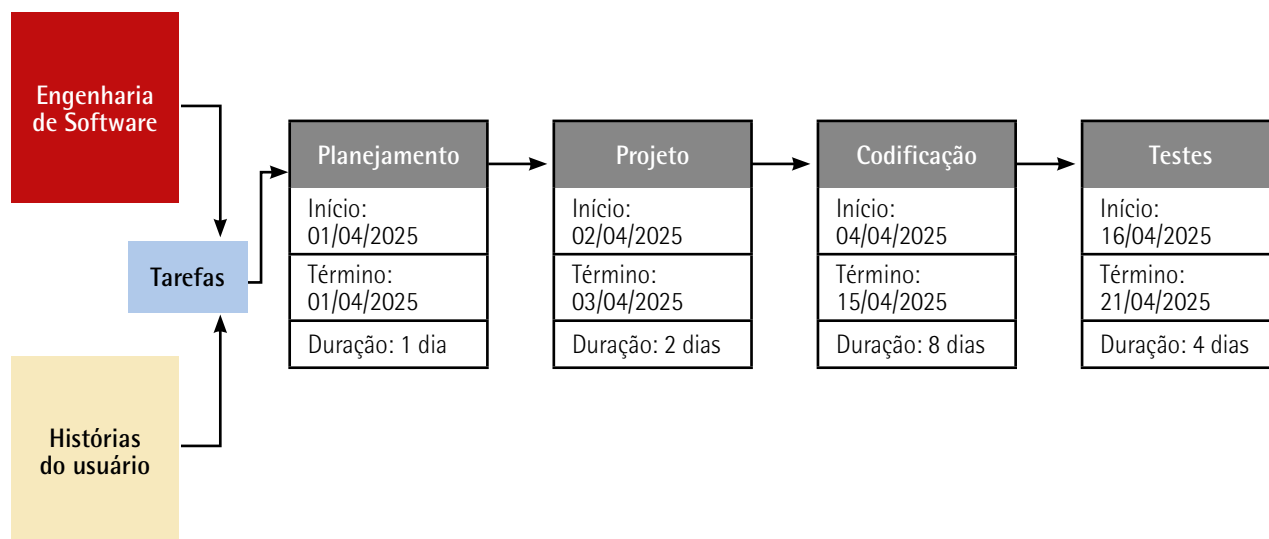


Figura 51 – Método Kanban aplicado à XP

7.3 Scrum

Scrum (o nome provém de uma atividade que ocorre durante a partida de rugby) é um método de desenvolvimento ágil de software bastante popular concebido por Jeff Sutherland e sua equipe de desenvolvimento no início dos anos 1990 (Pressman, 2021, p. 126).

O Scrum está estruturado nos valores e nos princípios do Manifesto Ágil, buscando proporcionar uma abordagem iterativa e incremental para o desenvolvimento de software.

Essa metodologia é projetada para atender às necessidades de projetos nos quais os requisitos são frequentemente instáveis ou desconhecidos, o que é típico em ambientes dinâmicos e imprevisíveis. Ao invés de um planejamento detalhado desde o início, o Scrum adota ciclos curtos e repetitivos, conhecidos como sprints, que permitem ajustes constantes e a entrega de valor incremental ao cliente.

O Scrum se destaca como um framework que combina práticas orientadas a objetos e princípios ágeis, promove a colaboração intensa entre desenvolvedores, stakeholders e o product owner (dono do produto), garantindo maior transparência e adaptabilidade ao longo do ciclo de desenvolvimento.



Saiba mais

Elaborado por seus criadores, Ken Schwaber e Jeff Sutherland, o Scrum guide é considerado a principal referência para a aplicação do Scrum. Esse guia define regras, papéis, eventos e artefatos do framework. Sua versão mais recente foi atualizada em novembro de 2020, reforçando os princípios do Scrum como um framework adaptável para diferentes contextos.

Você pode obtê-lo gratuitamente no site oficial:

Disponível em: <https://shre.ink/egv8>. Acesso em: 23 jun. 2025.

Observe a seguir as características da metodologia Scrum:

- **Equipes pequenas e multifuncionais:** ênfase na formação de equipes com competências diversas, com maior integração e que trabalham de forma autônoma e colaborativa, ideal para contextos complexos. Assim, obtém-se uma rápida solução de problemas.
- **Requisitos emergentes e pouco definidos:** os requisitos são mutáveis e críticos do negócio. No Scrum, o conhecimento é compartilhado de forma estratégica para enriquecer o trabalho em equipe e acelerar resultados. O foco da equipe é a capacidade de absorver mudanças e adaptar-se às condições do projeto de forma eficiente, promovendo uma cultura de colaboração contínua, na qual o progresso é medido continuamente com o mínimo de riscos.



Observação

Na dinâmica Scrum do rúgbi, os jogadores trabalham em conjunto, formando uma unidade coesa para mover a bola pelo campo. Essa metáfora é utilizada para descrever o espírito colaborativo e a proximidade das equipes na metodologia de desenvolvimento Scrum.

- **Sprints curtos e entregas frequentes:** no framework, o Scrum é um subconjunto do product backlog, que representa um ciclo de trabalho determinado regularmente para ter uma duração de 1 a 4 semanas. Assim, a equipe se concentra para realizar um conjunto de tarefas ou funcionalidades. O ciclo iterativo reduz riscos e oferece visibilidade contínua do progresso, possibilitando decisões informadas em tempo hábil. Nesse período curto de tempo, os desenvolvedores devem realizar uma série de atividades e a cada nova sprint, o profissional adquire novas experiências com base na etapa anterior, levando a melhorias em cada sprint subsequente. O número de sprints necessárias no product backlog varia conforme o tamanho e a complexidade do artefato a ser construído.

- **Product backlog:** é uma lista priorizada de requisitos ou características do artefato que agrega valor ao software, contendo funcionalidades, melhorias, correções de bugs, requisitos e tarefas que a equipe pode trabalhar. É gerenciado pelo product owner, que prioriza os itens com base no valor para o negócio e nas necessidades dos stakeholders.

O Scrum promove reuniões diárias de 15 minutos e todos respondem às questões:

- O que realizou hoje?
- Está tendo alguma dificuldade?
- O que irá fazer amanhã?



Observação

No esporte e em muitas outras atividades ocorrem as seguintes etapas: no início empregam-se muita energia e velocidade para conseguir uma boa posição; durante a competição, administra-se todo o desempenho; no fim, aplicam-se toda atenção, energia, potência e velocidade até a exaustão. O objetivo é conquistar a melhor posição e, se possível, a vitória. Essa finalização é chamada de sprint.

Observe os seguintes papéis desempenhados:

- **Agile coach:** orienta várias equipes Scrum, coordenando as interações entre as equipes e promovendo práticas ágeis.
- **Product owner:** responsável por maximizar o valor do produto e gerenciar o backlog. Prioriza as funcionalidades, mas não gerencia diretamente a equipe.
- **Scrum master:** é o facilitador das tarefas a serem realizadas pela equipe, removendo obstáculos que possam impedir seu progresso, promovendo a colaboração e a comunicação na realização dos objetivos da sprint.
- **Equipe de desenvolvimento:** conjunto de profissionais que trabalham na criação do produto, nas práticas de especificação, na modelagem, na programação, na análise de dados etc. Pode inclusive incorporar outras metodologias ágeis para atividades específicas, como XP, FDD e DevOps.

Observe como funciona o Scrum no framework ilustrado na figura 52. O processo ágil é organizado em ciclos definidos por breves períodos, acompanhados de certa disciplina em compasso com a sprint. No framework vemos um conjunto de ações de desenvolvimento do processo de software, aplicados de forma sequencial, com diversas iterações para alinhamento do conhecimento e acompanhamento do processo. No final da sprint ocorre a entrega do software, por incremento, e uma nova funcionalidade é demonstrada.

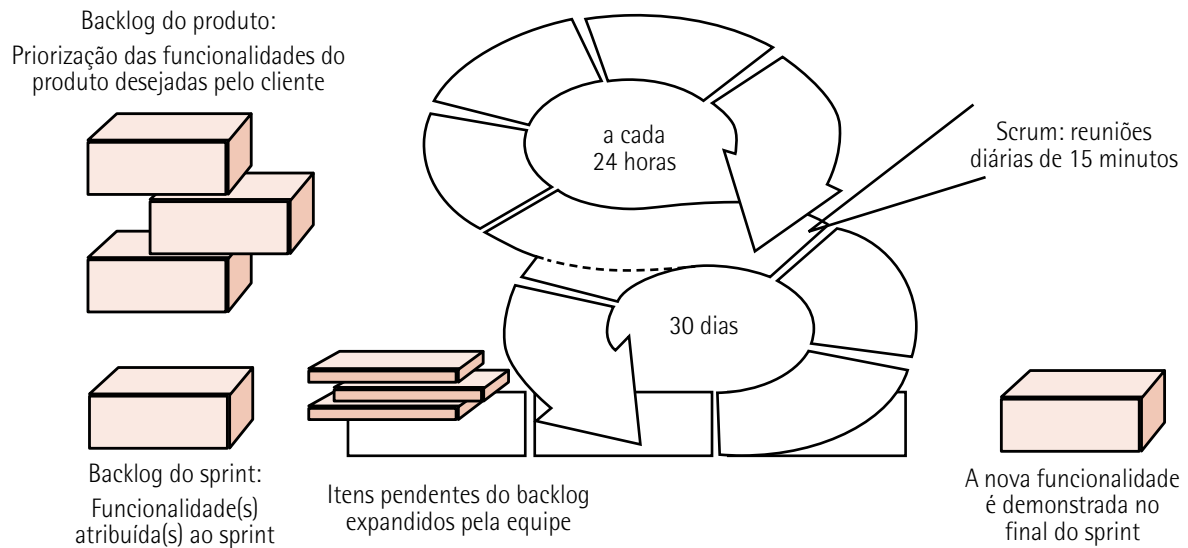


Figura 52 – Fluxo do processo Scrum

Adaptada de: Pressman (2021).

Acentuaremos a seguir as atividades estruturais da metodologia Scrum:

- **Planejamento**

- **Sprint planning:** reunião inicial de cada sprint na qual o time define o objetivo, as tarefas prioritárias e os critérios de aceitação para o ciclo.

- **Execução**

- **Daily Scrum:** reuniões rápidas diárias para sincronização, identificação de impedimentos e ajuste de planos, promovendo transparência.
- **Desenvolvimento de incrementos:** o time trabalha para construir funcionalidades ou soluções utilizáveis, baseando-se no backlog priorizado.

- **Monitoramento e controle**

- **Backlog refinement:** sessões para ajustar, detalhar ou repriorizar itens do backlog, garantindo alinhamento com as necessidades do produto.
- **Sprint review:** demonstração de incrementos concluídos para stakeholders, permitindo feedback e validação das entregas.

- **Reflexão e melhoria**

- **Sprint retrospective:** reunião de fechamento para refletir sobre o desempenho do time, identificar melhorias e implementar ações para sprints futuros.



Saiba mais

Explore mais a metodologia Scrum em:

Disponível em: <https://shre.ink/egvn>. Acesso em: 23 jun. 2025.

Um quadro Kanban pode ser integrado à metodologia Scrum, como é mostrado na figura 53. Com esse método, o ciclo de desenvolvimento do Scrum é feito a partir das etapas que se alinham ao Kanban.

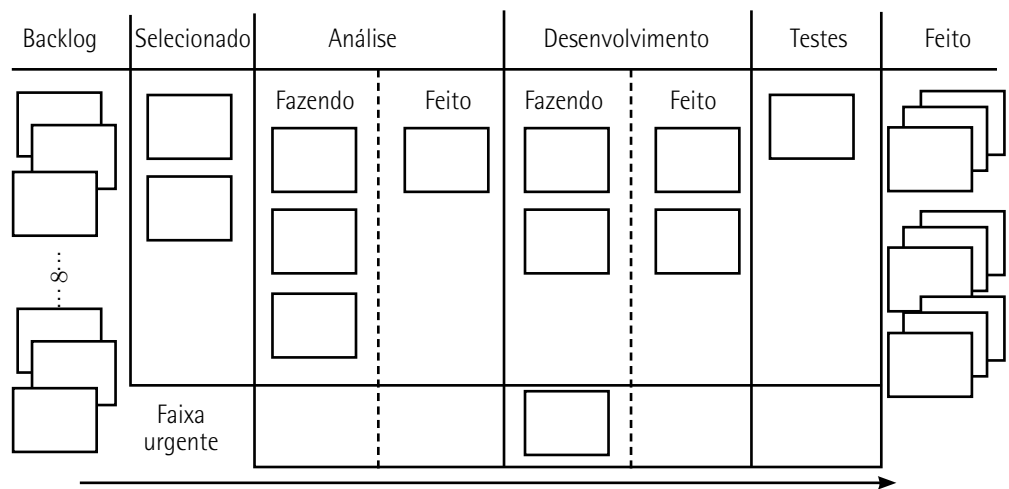


Figura 53 – Quadro Kanban

Adaptada de: Pressman (2021).

7.4 FDD

Feature Driven Development (FDD) é uma metodologia ágil que se destaca pela sua abordagem estruturada e orientada a objetos. O FDD é apropriado para equipes que desejam aprimorar a colaboração e aumentar a eficiência na entrega de projetos complexos.

Isso porque esta metodologia ágil enfatiza a importância de se dividir o desenvolvimento em características palpáveis, permitindo um progresso mensurável e contínuo.

Ou seja, ao entendê-la, os desenvolvedores e gestores de projeto podem descobrir novas maneiras de enfrentar desafios, otimizar workflows e alcançar objetivos com maior clareza e previsibilidade (CTC, 2024).

O FDD tem como foco a entrega incremental de funcionalidades visíveis e utilizáveis, sendo adaptado a projetos que exigem alto nível de adaptabilidade tecnológica.

Com o uso do FDD, os stakeholders têm acesso a entregas rápidas e relatórios de progresso estruturados em uma linguagem acessível, facilitando o alinhamento entre as expectativas de negócio e o desenvolvimento técnico.

Os gerentes de projeto se beneficiam de uma visibilidade abrangente e precisa do acompanhamento do projeto, com métricas objetivas que permitem tomadas de decisão ágeis e baseadas em dados.

Para os engenheiros de software, o modelo favorece ciclos curtos de desenvolvimento, possibilitando o início de novas funcionalidades em poucos dias e promovendo uma participação ativa em todas as etapas do processo, desde a análise de requisitos até o design e, consequentemente, a implementação de código.

No contexto do FDD, o termo central utilizado para controle e organização do projeto é chamado de característica (feature), uma unidade funcional de software que representa um comportamento do sistema que agrega valor perceptível para o usuário final.

Uma característica é uma funcionalidade granular, normalmente de pequena escala, que deve ser definida de forma colaborativa com o cliente ou as partes interessadas. Essa definição promove alinhamento entre os requisitos de negócio e a arquitetura técnica do sistema, permitindo que os engenheiros de software traduzam necessidades em implementações concretas.

A robustez da metodologia reside em sua combinação de práticas tradicionais de engenharia de software, como modelagem OO, com aspectos ágeis, como adaptação a mudanças e foco na colaboração contínua. Essa fusão permite um fluxo de trabalho adaptável e eficiente, garantindo alinhamento técnico e estratégico.

Observe a seguir os principais aspectos das características:

- As características são modeladas como unidades funcionais independentes ou pequenos blocos de funcionalidades, cuja definição considera o tamanho, o escopo e a complexidade da aplicação a ser desenvolvida. Isso permite maior rastreabilidade, mensuração de progresso e controle sobre o ciclo de desenvolvimento.
- Estruturalmente, essas características são agrupadas segundo critérios hierárquicos ou prioridades estratégicas do negócio, o que viabiliza a gestão eficiente do backlog técnico e o alinhamento com os objetivos organizacionais.
- A equipe de desenvolvimento trabalha com metas iterativas, comumente em ciclos quinzenais (timeboxes), com foco na entrega incremental de novas características. Essa abordagem impulsiona a entrega contínua de valor e promove práticas ágeis de engenharia, como integração contínua, revisão técnica frequente e validação incremental.

Observe agora o framework do FDD ilustrado na figura 54. A partir dos requisitos de software, iniciam-se a concepção e o planejamento, com o foco em criar um plano para a feature. A partir daí, em ciclos iterativos de desenvolvimento, é realizada a construção da feature, que faz parte do pacote de trabalho, gerando como resultado o software e os modelos de objetos, que podem ser reusados.

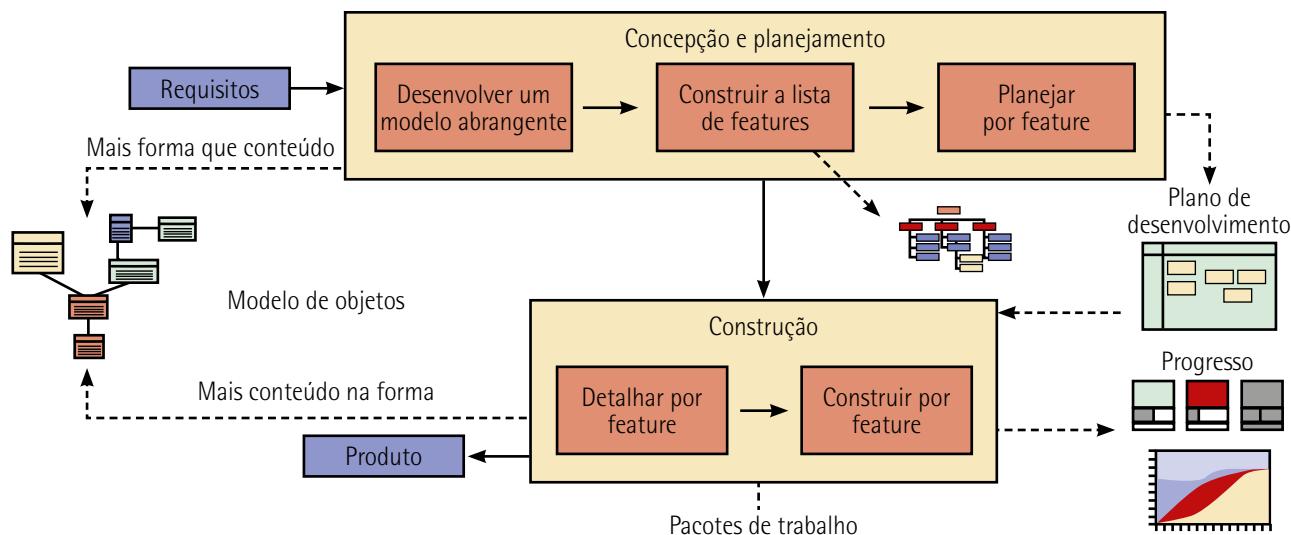


Figura 54 – Engenharia de software com FDD

O FDD é a metodologia ágil de desenvolvimento que mais enfatiza diretrizes e técnicas de gestão de projetos. O projeto é claro para todos os envolvidos e essa metodologia pode ser integrada com outras metodologias ágeis, como o Scrum. O FDD é composto por cinco processos principais (listados a seguir), que organizam e guiam o ciclo de desenvolvimento desde a modelagem inicial até a implementação de funcionalidades específicas. Ao final, a funcionalidade está pronta para entrega e avaliada.

- **Desenvolver um modelo geral:** criação de um modelo conceitual abrangente para entender os RS e criar uma visão de alto nível. Utilizam-se técnicas como modelagem de domínio, diagramas de classes e mapas conceituais. O objetivo é estabelecer a estrutura e os principais componentes do sistema antes de detalhar as funcionalidades.
- **Construir a lista de características:** com base no modelo desenvolvido, é criada uma lista hierárquica de funcionalidades pequenas, claras e orientadas ao negócio. Cada característica representa algo que o sistema deve fazer e é escrita em linguagem voltada ao cliente, como: "relatório de clientes". A lista é organizada em grupos chamados de grandes funcionalidades (major feature sets) e conjuntos de funcionalidades (feature sets).
- **Planejar por característica:** é estruturar o trabalho a partir de prioridades. Nesse processo, é feito o planejamento da implementação com base nas características identificadas. Atribuem-se responsabilidades como analistas de sistemas, programadores-chefe e outros e define-se a sequência de desenvolvimento. As funcionalidades são priorizadas com base em valor de negócio, dependências e complexidade.
- **Projetar por característica:** engloba a criação de designs específicos, quando cada funcionalidade é projetada detalhadamente antes da codificação. O programador-chefe seleciona a equipe que trabalhará na característica. É feita uma revisão de design envolvendo a equipe técnica, resultando em diagramas, documentos técnicos e decisões de arquitetura.

- **Construir por característica:** é o processo de codificação, teste e integração da característica ao sistema. Envolve implementação de código, testes unitários, integração contínua e revisão de código.

8 METODOLOGIAS ÁGEIS II

Neste tópico vamos trazer uma visão geral sobre algumas outras aplicações das metodologias ágeis, como DSDM, Crystal, AM e DevOps.

8.1 DSDM

É uma abordagem iterativa e incremental de CBSE (Component-Based Software Engineering – Engenharia de Software Baseada em Componentes) para o projeto de sistemas de software.

Criado no Reino Unido na década de 1990, o DSDM destacou-se por sua flexibilidade e seu foco na entrega rápida de soluções de alta qualidade, mesmo em cenários com prazos e orçamentos limitados. Ele é amplamente utilizado em projetos com escopos dinâmicos, permitindo adaptações rápidas às mudanças e garantindo que os objetivos sejam alcançados de forma prática e eficiente.

O DSDM surgiu como uma extensão do RAD. Com cronogramas e custos limitados, é aplicado em projetos de sistemas de software com foco em especificação, integração de seus componentes e testes para verificar se o sistema atende aos requisitos especificados.



Lembrete

Com diversas opções de metodologias ágeis, vale lembrar que a melhor aplicação de uma metodologia ágil é aquela que atende especificamente aos requisitos de negócio.



Saiba mais

O DSDM é particularmente útil em situações de componentização do software e que ajudam a manter o projeto no caminho certo, com limitações claras de tempo e orçamento.

Acesse o conteúdo indicado a seguir e conheça mais sobre essa metodologia ágil.

DSDM: tudo que você precisa saber sobre essa metodologia ágil. *FlowUp*, 13 fev. 2020. Disponível em: <https://shre.ink/ebpk>. Acesso em: 23 jun. 2025.

Os princípios fundamentais do DSDM são:

- **foco na necessidade do negócio:** priorizar entregas que agreguem valor ao cliente;
- **entrega dentro do prazo:** garantir que os projetos sejam concluídos de forma eficiente;
- **colaboração:** promover o trabalho conjunto entre equipes e stakeholders;
- **qualidade inegociável:** assegurar que os padrões de qualidade sejam mantidos;
- **desenvolvimento incremental e iterativo:** melhorar continuamente as entregas com base em feedbacks.

O framework do DSDM apresentado na ilustração da figura 55 é composto por oito fases principais, organizadas de forma cíclica e contínua.

- **Viabilidade (feasibility) – início do ciclo:** avalia se o projeto é tecnicamente viável e se faz sentido do ponto de vista do negócio. As ações se baseiam em fazer uma análise técnica preliminar, estimando inicialmente custos, tempo, recursos disponíveis e análise de riscos. Essa etapa inicial se refere à concepção do projeto, e somente depois dela se inicia o projeto.
- **Pré-projeto (pre-project):** garante que apenas projetos com uma base sólida e justificativa de negócios sejam iniciados. Seu objetivo é confirmar que existe uma justificativa clara para o projeto. Consiste nas práticas de análise inicial de viabilidade, definição de escopo em alto nível e identificação de stakeholders.
- **Fundamentos (foundations):** essa fase se preocupa em fixar uma base firme para o projeto antes de iniciar o desenvolvimento iterativo. As principais atividades são: definir a arquitetura, fazer a modelagem do processo de negócio, incluindo critérios de aceitação e plano de entrega.
- **Exploração (exploration):** nessa fase são desenvolvidos incrementos de solução que atendam aos requisitos priorizados. As principais atividades estão no desenvolvimento colaborativo com usuários, prototipagem rápida, iterações frequentes e feedbacks constantes.
- **Desenvolvimento iterativo (iterative development):** é o núcleo do DSDM. Essa fase busca garantir que a solução esteja tecnicamente robusta e pronta para produção. São realizados refino técnico, testes mais profundos e integração com sistemas de software existentes.
- **Implantação (deployment):** é a entrega da solução para uso real, garantindo que esteja pronta e validada pelos usuários. A fase de implantação consiste em treinamento, migração de dados, planejamento da transição e entrega do incremento.

- **Entrega frequente (frequent delivery):** refere-se à entrega contínua de versões executáveis do sistema em intervalos curtos, que variam entre um a três meses, permitindo obter feedback constante do cliente, reduzindo erros acumulados e mantendo o produto alinhado às necessidades do negócio.
- **Melhoria reflexiva (reflective improvement):** a equipe realiza pausas regulares para refletir sobre o processo de desenvolvimento, identificando pontos fracos e melhoria contínua. Promove um ciclo de aprendizado organizacional, elevando a maturidade da equipe e a qualidade do produto.
- **Segurança pessoal (personal safety):** garantias de que cada membro da equipe possa expressar suas opiniões, preocupações e ideias sem medo de retaliação. Assim, cria-se um ambiente psicológico seguro, que estimula a criatividade, a colaboração e a resolução de problemas de forma aberta e honesta.
- **Acesso fácil a usuários especialistas (easy access to expert users):** assegura que desenvolvedores tenham acesso direto e frequente a usuários finais ou especialistas no domínio do sistema. Esse acesso facilita o esclarecimento de dúvidas, o refinamento de requisitos e a validação de decisões em tempo real.
- **Testes automatizados (automated tests):** são incorporados testes automatizados como parte integrante do processo de desenvolvimento, de forma a garantir qualidade contínua do código, redução de regressões e permissões de refatoramento seguro, dando suporte ao desenvolvimento incremental e integração contínua.
- **Comunicação osmótica (osmotic communication):** envolve a comunicação informal e constante que ocorre naturalmente quando a equipe trabalha fisicamente próxima, compartilhando o mesmo ambiente, favorecendo a troca rápida de informações, as decisões eficientes e a redução da necessidade de reuniões formais.
- **Foco (focus):** as condições de trabalho devem favorecer a concentração e a produtividade, evitando interrupções desnecessárias, tarefas paralelas excessivas e ambientes ruidosos.



Saiba mais

Conheça mais a metodologia ágil Crystal, suas categorias e como ela pode ser aplicada na empresa em:

ENTENDA o que é a metodologia Crystal e saiba como ela pode ser aplicada no dia a dia das empresas. *Pontotel*, 5 out. 2023. Disponível em: <https://shre.ink/ebDo>. Acesso em: 23 jun. 2025.

8.3 AM

Modelagem Ágil (AM) consiste em uma metodologia prática, voltada para a modelagem e documentação de sistemas baseados em software. Simplificando, Modelagem Ágil consiste em um conjunto de valores, princípios e práticas voltados para a modelagem do software que podem ser aplicados a um projeto de desenvolvimento de software de forma leve e eficiente. Os modelos ágeis são mais eficientes do que os tradicionais pelo fato de serem simplesmente bons, pois não têm a obrigação de ser perfeitos (Pressman, 2016, p. 81).

A AM oferece uma abordagem estruturada, mas flexível, para representar e documentar software em projetos de desenvolvimento. Esse framework enfatiza a colaboração, a adaptabilidade e o uso de artefatos de modelagem que são "suficientemente bons" para atingir os objetivos do projeto ao invés de buscar uma perfeição detalhista, que pode se tornar obsoleta ou onerosa.

Ela apoia a criação de modelos que têm papel essencial na comunicação entre as equipes de desenvolvimento, stakeholders e designers; garantindo alinhamento técnico e funcional. Com uma abordagem iterativa e incremental, os modelos evoluem com o software, permitindo ajustes rápidos para atender às mudanças de requisitos e demandas do projeto.

A expressão "viajar leve" é central na filosofia AM. Ela orienta que apenas os modelos com valor prático e estratégico sejam mantidos a longo prazo, enquanto modelos temporários ou descartáveis são criados e eliminados conforme sua utilidade decresce. Essa abordagem apoia a engenharia de software moderna ao priorizar agilidade, redução de desperdícios e foco na entrega de valor contínuo.

O site oficial da Agile Modeling (AM, 2020) destaca alguns conceitos importantes:

Modelo: um modelo é uma abstração que expressa aspectos importantes de uma coisa ou conceitos. Os modelos podem ser visuais (diagramas), não visuais (descrições de texto) ou executáveis (código de trabalho ou equivalente). Às vezes, os modelos são chamados de mapas ou roteiros dentro da comunidade ágil.

Modelo ágil: Os modelos ágeis podem ser algo tão simples quanto adesivos em uma parede, esboços em um quadro branco, diagramas capturados digitalmente por meio de uma ferramenta de desenho ou modelos detalhados capturados usando uma ferramenta de engenharia de software baseada em modelo (MBSE – model-based software engineering).

Modelagem: modelagem é o ato de criar um modelo. A modelagem às vezes é chamada de mapeamento.

AM: é realizada de forma colaborativa e evolutiva.

Documento: um documento é uma representação persistente de uma coisa ou conceito. Um documento é um modelo, mas nem todos os modelos são documentos (a maioria dos modelos não é persistente).

Documento ágil: os documentos ágeis podem ser algo tão simples quanto notas pontuais, texto detalhado, testes executáveis ou um ou mais modelos ágeis.

Existe uma variedade de práticas ágeis e, de acordo com AM (2020), as práticas de design ágil vão desde práticas de arquitetura de alto nível até práticas de programação de baixo nível como é mostrado na figura 56. Cada uma dessas práticas é importante e cada uma é necessária para que sua equipe seja eficaz no design ágil. Em uma sequência de eventos, as práticas ágeis fazem a seguinte abordagem:

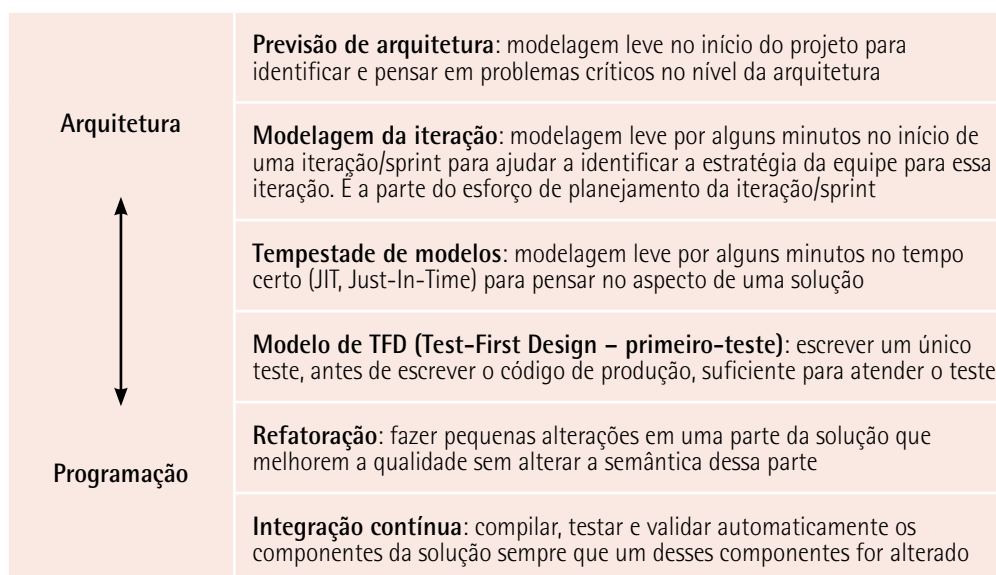


Figura 56 – Prática de design ágil



Saiba mais

Para obter mais informações sobre o AM, acesse:

Disponível em: <https://shre.ink/egCC>. Acesso em: 23 jun. 2025.

8.4 DevOps

O DevOps foi criado por Patrick DeBois para combinar Desenvolvimento (Development) e Operações (Operations). O DevOps tenta aplicar os princípios do desenvolvimento ágil e do enxuto a toda a cadeia logística de software (Pressman, 2021, p. 51).

A figura 57 mostra o fluxo de trabalho típico do DevOps, um paradigma amplamente adotado na engenharia de software para integrar equipes de desenvolvimento e operações em um ciclo contínuo de entrega de software.

Essa abordagem promove automação, colaboração e integração ao longo de diversas etapas essenciais, como planejamento, desenvolvimento, testes, integração contínua, entrega contínua, monitoramento e feedback. Elas são interligadas em um ciclo iterativo que permite ajustes rápidos e eficiência ao longo do desenvolvimento.

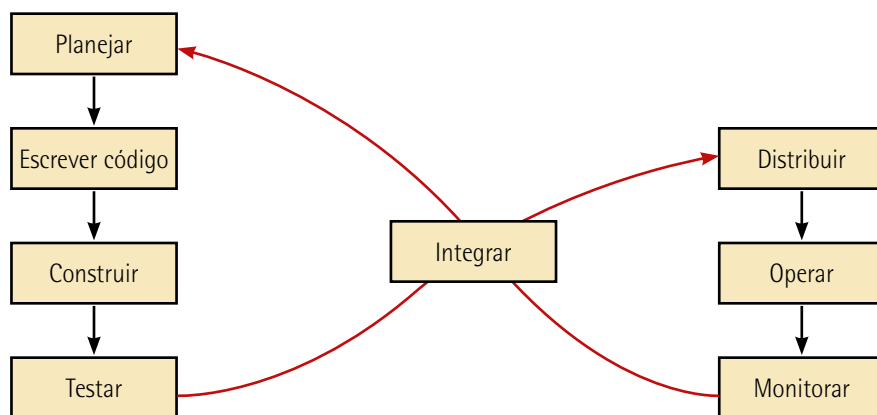


Figura 57 – Framework do DevOps

Adaptada de: Pressman (2021).

Do ponto de vista tecnológico, o DevOps faz uso de ferramentas especializadas, como Jenkins, Docker e Kubernetes, para automatizar processos críticos, desde a integração de código até o provisionamento e o monitoramento de infraestrutura. Elas são frequentemente utilizadas em conjunto: Jenkins para automação, Docker para containerização e Kubernetes para orquestração. Esse trio cria um fluxo de trabalho DevOps moderno, eficiente e resiliente.

Jenkins

É uma ferramenta de automação open-source focada em CI (Continuous Integration – Integração Contínua) e CD (Continuous Delivery – Entrega Contínua (CD)). Ela permite que equipes automatizem tarefas como construção, teste e implantação de software, reduzindo o trabalho manual e acelerando os ciclos de desenvolvimento.

Entre seus recursos, destacam-se: suporte a pipelines declarativos, integração com repositórios como Git e uma ampla biblioteca de plugins que permite personalizar e estender sua funcionalidade.

Exemplo de uso: sempre que um desenvolvedor realiza uma atualização no código Jenkins pode automaticamente compilar o código, executar testes e, se aprovado, implantar a aplicação no ambiente de staging ou produção.

Docker

É uma plataforma de containerização que permite empacotar aplicações e suas dependências em contêineres leves e portáteis. Ele garante que o software seja executado de forma consistente em diferentes ambientes do desenvolvimento, testes e produção, independentemente do sistema operacional ou da infraestrutura subjacente. Como benefícios, citam-se: redução de conflitos de dependências, maior escalabilidade e facilidade de migração de aplicações.

Exemplo de uso: uma equipe pode criar uma imagem Docker para sua aplicação web, garantindo que ela será executada da mesma maneira em um laptop de desenvolvimento ou em um cluster na nuvem.

Kubernetes (ou K8s)

É um sistema de orquestração de contêineres projetado para gerenciar grandes volumes de contêineres em ambientes distribuídos. O K8s automatiza tarefas complexas, como balanceamento de carga, escalonamento automático, gerenciamento de clusters e implantação contínua.

Tal dispositivo promove o controle por escalonamento automático (autoscaling), permitindo que uma funcionalidade de software se ajuste dinamicamente à infraestrutura de software, ambientes de computação em nuvem e a recursos disponíveis de acordo com a demanda do sistema. Também permite a verificação contínua do status e do funcionamento correto dos componentes da aplicação com reinício automático de contêineres defeituosos.

Exemplo de uso: uma organização pode usar Kubernetes para implantar e gerenciar microsserviços em um cluster de servidores na nuvem, garantindo alta disponibilidade e recuperação automática em caso de falhas.

O DevOps tem como objetivo otimizar a entrega de valor ao cliente por meio de ciclos curtos e iterativos, garantindo flexibilidade para atender às mudanças de requisitos e necessidades do mercado, bem como às metodologias Scrum e XP, que também enfatizam a colaboração, a comunicação eficaz entre equipes multifuncionais e o aprendizado contínuo.

Exemplo de aplicação

Caso: monitoramento e atualização constante no e-commerce.

Uma plataforma de e-commerce precisa ser constantemente atualizada para incluir novos recursos, corrigir bugs e lidar com picos de tráfego durante promoções sazonais.

Etapas do workflow

1. **CI**: desenvolvedores trabalham em diferentes partes do código e enviam as alterações para um repositório compartilhado, como o Git. Com a ajuda de ferramentas como Jenkins ou GitHub Actions, o

código é automaticamente integrado, testado e construído; garantindo que novas implementações não comprometam o sistema existente.

2. CD: após a passagem do código nos testes automáticos, o código é automaticamente empacotado em contêineres Docker e implantado em um ambiente de pré-produção para validação final.

3. IaC (Infrastructure as Code – Infraestrutura como Código): a infraestrutura de hospedagem da plataforma (servidores, balanceadores de carga e bancos de dados) é gerenciada por meio de scripts usando ferramentas como Terraform ou AWS CloudFormation, permitindo replicar o ambiente em minutos.

4. Monitoramento contínuo: uma vez em produção, ferramentas como Prometheus e Grafana monitoram o desempenho da aplicação, verificando métricas como latência, uso de CPU e comportamento de banco de dados. São configurados alertas para identificar problemas em tempo real, como quedas no desempenho ou falhas nos serviços.

5. Autoscaling e resiliência: durante grandes eventos do comércio como Black Friday e períodos de fim de ano, em que há uma demanda maior no e-commerce, o Kubernetes gerencia o escalonamento automático de pods (componentes de implantação) para lidar com o aumento do tráfego. Caso ocorra uma falha no pod, ele é automaticamente reiniciado para evitar interrupções no serviço.

8.5 Métodos, ferramentas e técnicas: aplicativos para operações com metodologias ágeis

Normalmente, em metodologias ágeis, o gerenciamento de projetos e organização de tarefas é baseado em sistema visual de quadros, listas e cartões; acompanhado de regras que simplificam processos repetitivos, como mover cartões automaticamente ao serem concluídos.



Observação

Cada backlog pode ser refinado e dividido em tarefas menores durante o planejamento da sprint, garantindo que a equipe Scrum mantenha foco e transparência.

Cada sprint pode ser organizada em um quadro específico, permitindo que a equipe acompanhe o progresso de forma colaborativa.

Nessa categoria de serviços, exploraremos uma ferramenta de software que atenda a essas necessidades de serviços.

Existem várias ferramentas, a exemplo do Trello, uma ferramenta visual e intuitiva que facilita o gerenciamento de projetos e a organização de tarefas, sendo altamente eficiente quando aplicado à metodologia Scrum.

Os quadros do Trello representam o projeto, as listas são usadas para organizar o fluxo das tarefas e os cartões representam as histórias ou tarefas individuais e que podem conter informações detalhadas.

Os principais recursos do Trello são:

- **Quadros:** representam projetos ou áreas de trabalho.
- **Listas:** organizam tarefas em diferentes etapas, como "a fazer", "em progresso" e "concluído".
- **Cartões:** contêm tarefas específicas, que podem incluir descrições, verificações (checklists), prazos, anexos e comentários.
- **Automação:** com o recurso Butler, é possível automatizar fluxos de trabalho repetitivos.
- **Integrações:** pode-se conectar a outras ferramentas, como Slack, Google Drive e Jira.
- **Colaboração:** permite que equipes trabalhem juntas em tempo real, com notificações e atualizações instantâneas.

A figura 58 acentua os quadros Trello, templates e outras funções. São diversas as aplicações na gestão de tarefas e equipes, para os mais diversos processos e métodos da engenharia de software. Os templates são modelos úteis para diversos tipos de negócios, como design, engenharia, gestão de projetos e produtividade.

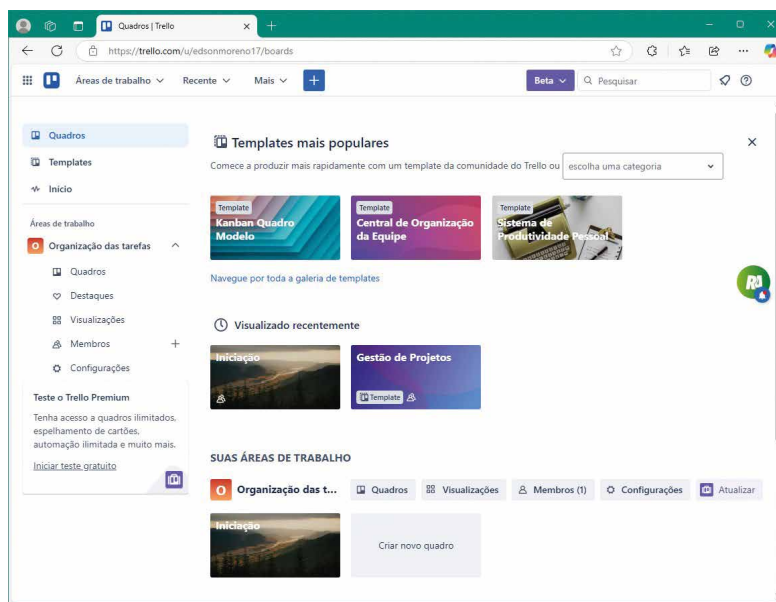


Figura 58 – Quadros Trello

Disponível em: <https://shre.ink/ebkn>. Acesso em: 23 jun. 2025.



Saiba mais

Para conhecer melhor o assunto tratado, acesse o site oficial da Trello:

Disponível em: <https://shre.ink/egCe>. Acesso em: 23 jun. 2025.

Exemplo de aplicação

Caso: Scrum, organização de backlogs e sprints em Kanban com uso da ferramenta Trello.

Aplicação: implementação em software IoT utilizando a metodologia Scrum.

A seguir foram selecionados alguns backlogs para a implementação de um software IoT. Use o Trello e acompanhe as operações a seguir:

1. Conexão de dispositivos

- implementar APIs para conexão entre dispositivos IoT;
- configurar protocolos de comunicação (MQTT ou HTTP).

2. Coleta e armazenamento de dados

- projetar bancos de dados para dados coletados;
- otimizar armazenamento em nuvem ou local;
- codificar e implementar app.

3. UX e UI

- melhorar interface para usuários administrarem dispositivos;
- testar usabilidade com clientes reais.

No início, escolha um template para um negócio específico e monte o quadro de cartões. O modelo de cartões dos backlogs apresentados na figura 59 foi criado com o nome "Modelo Scrum – Kanban". Como entrada de dados, essa é a primeira etapa para a distribuição de períodos e responsabilidades.

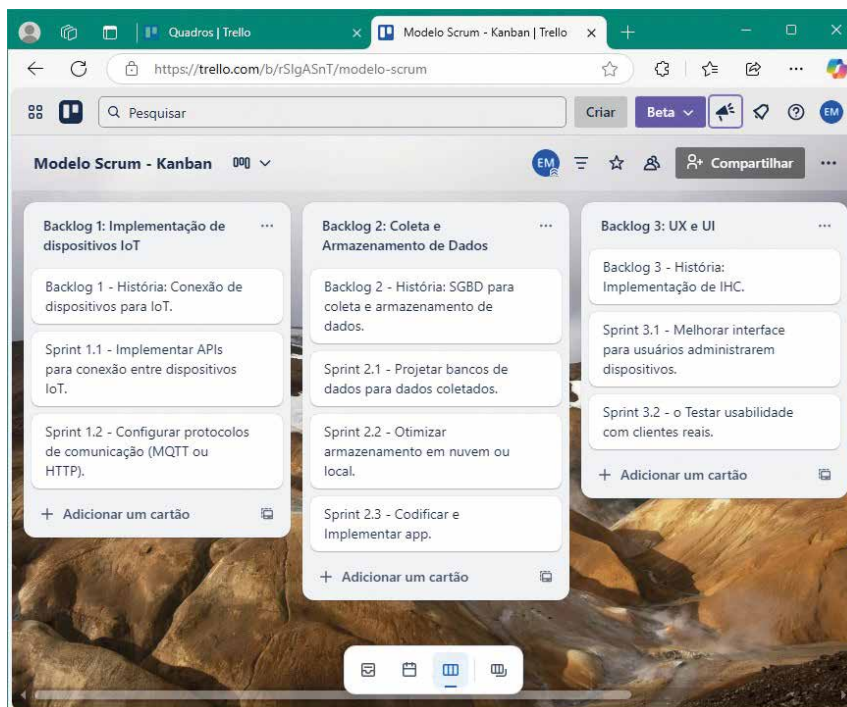


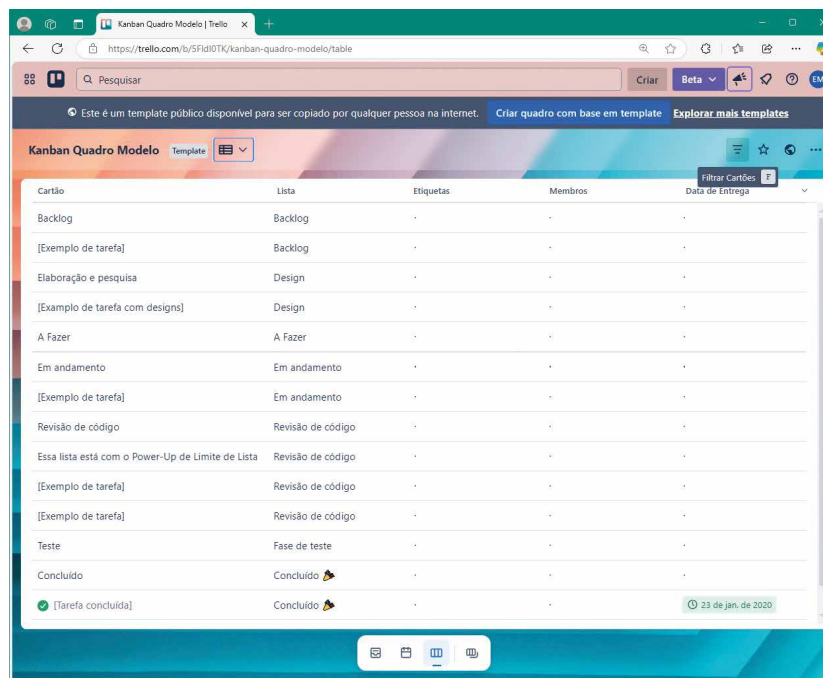
Figura 59 – Cartões "Modelo Scrum – Kanban"

Após a operação inicial, o desenvolvedor pode configurar a sua forma com diversas funções mostradas na figura 60.



Figura 60 – Cartões "Modelo Scrum – Kanban"

Com o template "Kanban Quadro Modelo", é possível associar o backlog com seu respectivo status e sequência pelos cartões: "Backlog", "Design", "A Fazer", "Em andamento", "Revisão de código", "Fase de teste" e "Concluído" e outros que queira criar. Veja como fica a tela de operação "Tabela", mostrada na figura 61 a seguir.



Cartão	Lista	Etiquetas	Membros	Data de Entrega
Backlog	Backlog	-	-	-
[Exemplo de tarefa]	Backlog	-	-	-
Elaboração e pesquisa	Design	-	-	-
[Exemplo de tarefa com designs]	Design	-	-	-
A Fazer	A Fazer	-	-	-
Em andamento	Em andamento	-	-	-
[Exemplo de tarefa]	Em andamento	-	-	-
Revisão de código	Revisão de código	-	-	-
Essa lista está com o Power-Up de Limite de Lista	Revisão de código	-	-	-
[Exemplo de tarefa]	Revisão de código	-	-	-
[Exemplo de tarefa]	Revisão de código	-	-	-
Teste	Fase de teste	-	-	-
Concluído	Concluído 🏆	-	-	-
✅ [Tarefa concluída]	Concluído 🏆	-	-	🕒 23 de jan. de 2020

Figura 61 – Trello: Template "Kanban Quadro Modelo", função Tabela

Na figura 62 vemos a função Painel, que mostra os níveis de eficácia das tarefas em andamento naquele instante.

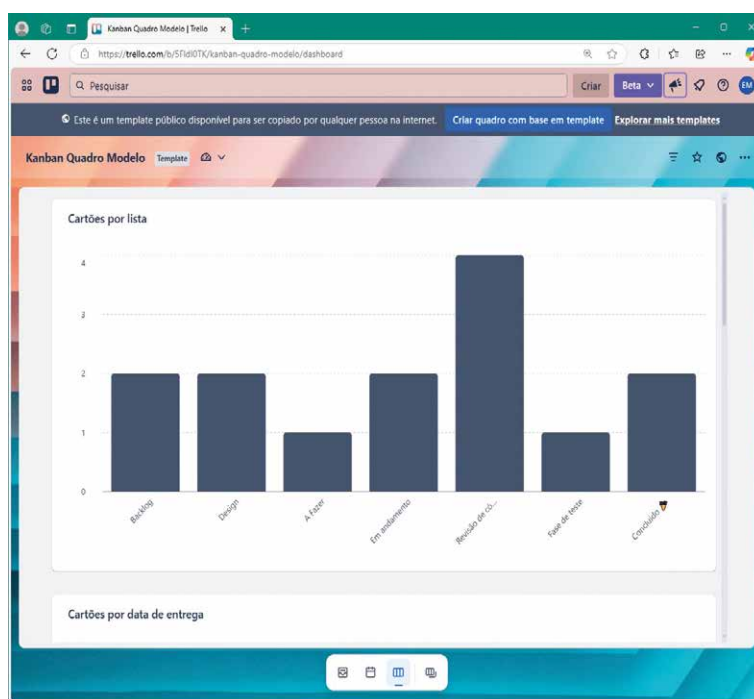


Figura 62 – Trello: Template "Kanban Quadro Modelo", função Painel

Enfim, o uso da ferramenta Trello, aplicada com a metodologia Scrum, promove transparência, agilidade e organização; tornando o gerenciamento de projetos mais visual e acessível para todos os membros da equipe. Facilita a comunicação entre equipes por meio de quadros, listas e cartões intuitivos, personalizados para diferentes metodologias. Como se adapta a necessidades específicas, integra-se com outras plataformas, com recursos de automação de tarefas repetitivas, otimizando o tempo e aumentando a produtividade de software.



Resumo

Esta unidade aborda os fundamentos, as práticas e as ferramentas relacionadas às metodologias ágeis de desenvolvimento de software, dividindo-se em duas partes principais, as metodologias ágeis I e II.

Inicialmente, destacamos o Manifesto Ágil, que estabelece valores e princípios orientados no desenvolvimento ágil de software como a valorização da colaboração com o cliente, a entrega contínua de software funcional e a rápida resposta às mudanças. Em seguida, apresentamos as vantagens e as desvantagens entre os modelos estruturados, como RUP e os modelos ágeis, com destaque para o Scrum.

Na sequência, foram acentuadas metodologias ágeis específicas que se destacam no mercado entre as dezenas existentes, como XP, cujo foco envolve técnicas rigorosas, como programação em par, integração contínua e testes automatizados; e Kanban, um método visual de gerenciamento de tarefas por meio de cartões para melhorar o fluxo de trabalho contínuo.

Vimos que o baixo uso de algumas metodologias ágeis ocorre porque elas são metodologias mais específicas, porém de grande uso e aplicabilidade, como é o caso do FDD, um processo orientado por funcionalidades com ênfase em planejamento incremental e design baseado em modelos.

A segunda parte (metodologias ágeis II) explora outras metodologias ágeis específicas e completas para diversos processos e atividades do desenvolvimento de software e que podem ser aplicadas isoladamente ou integradas a outros modelos de processos estruturados ou metodologias ágeis.

Nesse contexto, acentuamos o DSDM, uma extensão do modelo de processo RAD. É um framework iterativo e incremental para entrega rápida de soluções de alta qualidade com orçamentos limitados. Também citamos a família Crystal de metodologias ágeis, fortemente centrada nas relações humanas, comunicação e simplicidade, elementos considerados ajustáveis conforme o tamanho e a criticidade do projeto.

Prosseguindo nossos estudos, elencamos as ferramentas e as técnicas de AM, um conjunto de práticas que promove a simplicidade e a comunicação sob o princípio de "viajar leve", que orienta apenas manter os modelos com valor prático e estratégico.

As metodologias ágeis trouxeram eficiência na forma de produzir software e a cada dia surgem técnicas e frameworks que prometem ainda mais melhorar essa eficiência, como é o caso do DevOps, que integra desenvolvimento e operações para automatizar processos e acelerar entregas com qualidade.

Por fim, estudamos a ferramenta Trello, com recursos de criar quadros, listas, cartões, automação, integrações e colaboração. Ela é amplamente utilizada nas metodologias ágeis Scrum e Kanban; tornou-se popular devido à sua interface intuitiva e flexível, apropriada para equipes menores ou projetos menos complexos.



Exercícios

Questão 1. (UFU-MG 2023, adaptada) Metodologias de desenvolvimento de software chamadas de ágeis são baseadas em desenvolvimento iterativo, no qual requisitos e soluções evoluem pela colaboração entre equipes auto-organizadas. Essas metodologias encorajam frequentes inspeção e adaptação, alinhamento entre o desenvolvimento e os objetivos dos clientes e um conjunto de boas práticas que permita entregas rápidas e de qualidade.

Considerando as metodologias ágeis de desenvolvimento de software, avalie as afirmativas.

I – O Scrum adota uma abordagem empírica, pois entende que o problema pode não ser totalmente compreendido nem estar totalmente definido na análise e que os requisitos podem mudar com o passar do tempo. Essa abordagem mantém o foco em maximizar a habilidade da equipe em responder de forma ágil aos desafios emergentes.

II – Um dos valores do método XP é a busca pela simplicidade, de modo que o software deve ser mantido o mais simples possível pelo maior tempo possível.

III – O TDD é uma prática que envolve pequenas iterações, assim, novos casos de teste são escritos considerando uma melhoria ou uma nova funcionalidade. O código necessário é implementado para atender a esse teste.

É correto o que se afirma em:

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa E.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: o Scrum é uma metodologia ágil que enfatiza a adaptabilidade da equipe de desenvolvimento às mudanças. Ele é aplicado a equipes que têm o foco em construir e integrar softwares com

requisitos pouco estáveis, desconhecidos ou que mudam com frequência. Ao adotarmos esse método, devemos reconhecer que os requisitos podem evoluir ao longo do tempo e que a compreensão do problema pode ser aprimorada à medida que o projeto avança.

II – Afirmativa correta.

Justificativa: no método XP, são incentivadas a colaboração estreita entre clientes e desenvolvedores e a produção de documentações simplificadas. São projetadas apenas as necessidades imediatas, com projetos simples e de fácil implementação de código.

III – Afirmativa correta.

Justificativa: o TDD consiste na conversão de requisitos de software em testes, o que ocorre antes de o projeto ser concluído. No TDD, casos de teste são escritos antes da implementação do código. O código é desenvolvido para atender a esses testes, promovendo o desenvolvimento iterativo e a melhoria contínua do software.

Questão 2. (FGV 2024, adaptada) O Time de Soluções Inovadoras (TISI) de uma organização está utilizando práticas do Kanban no processo de desenvolvimento de soluções de software.

Com o uso do Kanban, o TISI visa:

A) Utilizar uma abordagem de mudança evolutiva, baseada em feedbacks, para o processo de desenvolvimento de soluções de software.

B) Manter um workflow padrão para o processo de inovação.

C) Gerenciar e melhorar o desempenho dos indivíduos do time.

D) Estabelecer uma metodologia de desenvolvimento de soluções de software de modo iterativo.

E) Aplicar um processo para definição de incrementos para desenvolvimento de soluções de software.

Resposta correta: alternativa A.

Análise das alternativas

A) Alternativa correta.

Justificativa: Kanban é um método ágil de gestão visual de trabalho que ajuda a otimizar fluxos, reduzir desperdícios e melhorar continuamente seus processos. Nesse método, é utilizado um quadro Kanban, que é dividido em colunas que representam os estágios do fluxo de trabalho. Cada tarefa é indicada por um cartão que avança entre as colunas. A essência do Kanban inclui a mudança evolutiva

(não disruptiva), a melhoria contínua baseada em feedback, a adaptação gradual do processo e o foco na otimização do fluxo de trabalho.

B) Alternativa incorreta.

Justificativa: o Kanban tem o objetivo de visualizar e melhorar o fluxo de trabalho, não necessariamente padronizá-lo.

C) Alternativa incorreta.

Justificativa: o Kanban foca no fluxo de trabalho do sistema como um todo, seu objetivo principal não é a avaliação individual.

D) Alternativa incorreta.

Justificativa: o Kanban não estabelece iterações fixas como no Scrum. O fluxo no Kanban é contínuo, não iterativo.

E) Alternativa incorreta.

Justificativa: o método Kanban não trabalha com incrementos predefinidos. A descrição do texto da alternativa aproxima-se mais do Scrum que do Kanban.

REFERÊNCIAS

ABNT. *NBR ISO 9241-11: ergonomia da interação humano-sistema. Parte 11: usabilidade: definições e conceitos*. 2. ed. Rio de Janeiro, 2021.

ABNT. *NBR ISO 9241-210: ergonomia da interação humano-sistema. Parte 210: projeto centrado no ser humano para sistemas interativos*. 2. ed. Rio de Janeiro, 2024.

ABNT. *NBR ISO/IEC 9241-11: requisitos ergonômicos para trabalho de escritórios com computadores*. Rio de Janeiro, 2002.

AMBLER, J. *et al.* Including digital sequence data in the Nagoya Protocol can promote data sharing. *Trends in biotechnology*, v. 39, n. 2, p. 116-125, 2021.

AMBYSOFT. *Effective practices for software solution delivery*. [s.d.]. Disponível em: <https://shre.ink/xEWy>. Acesso em: 23 jun. 2025.

5 APLICAÇÕES de realidade aumentada nos SIG. *INFO Portugal*, 30 out. 2020. Disponível em: <https://shre.ink/egGU>. Acesso em: 23 jun. 2025.

ASTAH. *Powering engineering excellence with Astah*. [s.d.]. Disponível em: <https://shre.ink/egKD>. Acesso em: 23 jun. 2025.

BASTOS, A. Os tipos de metodologias ágeis mais usados pelas empresas. *Alura para Empresas*, 3 maio 2023. Disponível em: <https://shre.ink/ebdg>. Acesso em: 23 jun. 2025.

BECK, K. *et al.* Manifesto para Desenvolvimento Ágil de Software. *Agile Manifesto*, 2001. Disponível em: <https://shre.ink/ebdl>. Acesso em: 23 jun. 2025.

BOOCH, G. *UML: guia do usuário*. São Paulo: Campus, 2002.

CAMARGO, R. Extreme Programming: quais principais regras e valores? *Robson Camargo Projetos e Negócios*, 3 out. 2019. Disponível em: <https://shre.ink/ebWL>. Acesso em: 23 jun. 2025.

CAPUTO, K.; HEFNER, R. *Strategies for transitioning from SW-CMM to CMMI*. Flórida: DELTA Business Solutions, 2004. Disponível em: <https://shre.ink/ebS9>. Acesso em: 23 jun. 2025.

CMMI. *CMMI(R) para desenvolvimento: versão 1.2*. Pittsburgh: Carnegie Mellon – Software Engineering Institute, 2006.

COSTA, O. W. D. *JAD: Joint Application Design*. Rio de Janeiro: IBPI Press, 1994.

DELUCA, J. A importância da engenharia de software: FDD Feature-Driven Development. *Engenheiros do Software*, 7 out. 2008. Disponível em: <https://shre.ink/ebBp>. Acesso em: 23 jun. 2025.

DIJKSTRA, E. W. The humble programmer [1972 ACM Turing Award Lecture]. *Communications of the ACM*, v. 15, n. 10, p. 859-866, 1972.

DSDM: tudo que você precisa saber sobre essa metodologia ágil. *FlowUp*, 13 fev. 2020. Disponível em: <https://shre.ink/ebpk>. Acesso em: 23 jun. 2025.

ECLIPSE process framework project. *Eclipse Foundation*, [s.d.]. Disponível em: <https://shre.ink/ebp8>. Acesso em: 23 jun. 2025.

ENTENDA o que é a metodologia Crystal e saiba como ela pode ser aplicada no dia a dia das empresas. *Pontotel*, 5 out. 2023. Disponível em: <https://shre.ink/ebDo>. Acesso em: 23 jun. 2025.

ENTENDENDO o Feature Driven Development (FDD) no desenvolvimento ágil. *CTC*, 22 abr. 2024. Disponível em: <https://shre.ink/ebBz>. Acesso em: 23 jun. 2025.

ENTERPRISE Unified Process (EUP): strategies for enterprise Agile. *Ambsoft Inc*, [s.d.]. Disponível em: <https://shre.ink/xEpP>. Acesso em: 23 jun. 2025.

FOURNIER, R. *Desenvolvimento e manutenção de sistemas estruturados*. São Paulo: Makron Books, 1994.

FOWLER, M. *UML essencial: um breve guia para a linguagem padrão de modelagem de objetos*. 2. ed. Porto Alegre: Bookman, 2000.

GUEDES, G. T. A. *UML 2: guia prático*. São Paulo: Novatec Editora, 2007.

HAYES, W.; OVER, J. W. The personal software process (PSP): an empirical study of the impact of PSP on individual engineers. Pensilvânia (EUA): CMU/SEI, 1997. Disponível em: <https://shre.ink/ebf3>. Acesso em: 23 jun. 2025.

HUMPHREY, W. S. *A discipline for software engineering*. Massachusetts (EUA): Addison-Wesley, 1995.

HUMPHREY, W. S. *The Team Software Process (TSP)*. Pensilvânia (EUA): CMU/SEI, 2000.

IBM. *Rational software architect standard edition*. [s.d.]. Disponível em: <https://shre.ink/egum>. Acesso em: 23 jun. 2025.

ICONIX. *Sparx Systems*, [s.d.]. Disponível em: <https://shre.ink/egDa>. Acesso em: 23 jun. 2025.

ISO. 9241-11:2018. Ergonomics of human-system interaction. Part 11: usability: definitions and concepts. Genebra (Suíça), 2021a. Disponível em: <https://shre.ink/ebIE>. Acesso em: 23 jun. 2025.

ISO. 9241-20:2021. Ergonomics of human-system interaction. Part 20: an ergonomic approach to accessibility within the ISO 9241 series. Genebra (Suíça), 2021b. Disponível em: <https://shre.ink/ebIf>. Acesso em: 23 jun. 2025.

KRUCHTEN, P. *The rational unified process: an introduction*. 2. ed. Massachusetts (EUA): Addison-Wesley, 2000.

LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao processo unificado*. 2. ed. Porto Alegre: Bookman, 2007.

LAUDON, K. C.; LAUDON, J. P. *Sistemas de informação gerenciais*. 11. ed. São Paulo: Bookman, 2014.

LAUDON, K. C.; LAUDON, J. P. *Sistemas de informação gerencial: administrando a empresa digital*. 17. ed. São Paulo: Bookman, 2023.

MARTINS, J. V. *Desenvolvimento de protótipos de interfaces humano-computador para uma funcionalidade do Moodle para convergência digital*. 2011. Trabalho de Conclusão de Curso (Sistemas de Informação) – Universidade Federal de Santa Catarina, Santa Catarina, 2011.

A MISSÃO da Modelagem Ágil. Agile Modeling, 2020. Disponível em: <https://shre.ink/xLBh>. Acesso em: 4 jul. 2025.

MODELAGEM de processos de negócios com Bizagi. Bizagi, 2024. Disponível em: <https://shre.ink/eghM>. Acesso em: 23 jun. 2025.

NUNES, M. *Fundamental de UML*. 2. ed. Lisboa: Editora de Informática, 2010.

PAKISTAN INSTITUTE OF ENGINEERING AND APPLIED SCIENCES (PIEAS). *Software development methodologies: computer science*. Islamabad (Paquistão): Pieas, 2017. Disponível em: <https://shre.ink/eb88>. Acesso em: 23 jun. 2025.

PATEL, N. Teste de usabilidade: o que é e como fazer passo a passo. NEILPATEL, 2018. Disponível em: <https://shre.ink/ebr2>. Acesso em: 23 jun. 2025.

PAULA FILHO, W. P. *Engenharia de software: fundamentos, métodos e padrões*. 3. ed. Rio de Janeiro: LTC, 2015.

PAULK, M. C. et al. *The capability maturity model for software: version 1.1*. Pittsburgh (EUA): Software Engineering Institute Customer Relations, 1993.

PFFLEGER, S. L. *Engenharia de software*. 2. ed. São Paulo: Prentice Hall, 2004.

PRAXIS. *Visão geral*. 2023. Disponível em: <https://shre.ink/egKn>. Acesso em: 23 jun. 2025.

PRESSMAN, R. S. *Engenharia de software*. 5. ed. Rio de Janeiro: McGraw-Hill, 2002.

PRESSMAN, R. S. *Engenharia de software*. 6. ed. Rio de Janeiro: McGraw-Hill, 2007.

PRESSMAN, R. S. *Engenharia de software: uma abordagem profissional*. 7. ed. Rio de Janeiro: McGraw-Hill, 2011.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.

PROJECT MANAGEMENT INSTITUTE (PMI). *PMBOK: um guia do conhecimento em gerenciamento de projetos*. 4. ed. Atlanta (EUA): PMI, 2010.

PROJECT MANAGEMENT INSTITUTE (PMI). *PMBOK: guia do conhecimento em gerenciamento de projetos*. 6. ed. Atlanta (EUA): PMI, 2017.

PROJECT MANAGEMENT INSTITUTE (PMI). *PMBOK: guia do conhecimento em gerenciamento de projetos*. 7. ed. Atlanta (EUA): PMI, 2021.

ROBASKI, J. E.; RAMOS, E. FDD (Feature Driven Development). *Medium*, 25 ago. 2014. Disponível em: <https://shre.ink/xEeQ>. Acesso em: 23 jun. 2025.

SOMMERVILLE, I. *Engenharia de software*. 8. ed. São Paulo: Pearson Prentice Hall, 2003.

SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SOMMERVILLE, I. *Software engineering*. 10. ed. Edimburgo (Escócia): Pearson Education Limited, 2016.

SOUZA, C. S. et al. *Projeto de interfaces de usuário: perspectivas cognitivas e semióticas*. Rio de Janeiro: PUC, 1999.

STAIR, R. M.; REYNOLDS, G. W. *Princípios de sistemas de informação: uma abordagem gerencial*. São Paulo: Pioneira Thomson Learning, 2006.



Handwriting practice lines consisting of 30 horizontal blue lines. Each line is preceded by a small blue dot, serving as a guide for letter height and placement.



Informações:
www.sepi.unip.br ou 0800 010 9000